

BIND: Binding 3D Robot Actions to 2D Image Features

Cameron Smith¹, Arsh Tangri², Vitor Guizilini², Yue Wang¹, Zubair Irshad², Sergey Zakharov²

Abstract—We introduce BIND, a new action representation for visuomotor robot policies that binds 3D robot actions to their corresponding 2D image features, yielding strong data efficiency gains and robustness to out-of-distribution object positions and camera viewpoints. The action heads of current robot policies are typically formulated as an MLP regression from a single global feature vector produced by a pretrained vision encoder. This global formulation requires the policy network to discover, from demonstrations alone, the relationship between target robot actions and the image features they project onto. The consequence is that although modern image features are semantically descriptive, spatially robust, and even multiview-consistent, the policies built on them are brittle to subtle changes in camera viewpoint and object placement—and surprisingly data-inefficient. BIND closes this gap by supplying the action–feature relationship through camera geometry rather than learning: it discretizes a volume of candidate end effector positions, attaches each candidate to the pretrained features at its projection in each camera view, and selects actions by scoring each candidate’s position and image-bound feature combination. On a real robot, we study data efficiency and out-of-distribution robustness to unseen object positions and camera viewpoints, as well as general long-horizon task execution and dexterity. We find BIND to be highly data-efficient and robust: it achieves near-perfect success on tasks with as few as 5 demonstrations, and degrades gracefully under steep camera-viewpoint shifts and held-out object positions where coordinate-regression baselines completely fail.

I. INTRODUCTION

Behavioral cloning—training a robot policy as a supervised mapping from observations to actions on demonstration data—is typically instantiated by adding an MLP head that regresses robot actions from the globally pooled feature vector of a pretrained image encoder. The encoders themselves are strong: recent studies demonstrate that modern image features are semantically descriptive, spatially consistent, and even multiview-consistent, and that simple zero-shot probes recover geometry and correspondence from them. The policies built on these features, however, remain brittle even on simple tasks: small changes in camera pose, lighting, or object placement relative to the training distribution frequently cause complete task failure, despite the underlying features being largely invariant to such shifts. Robot policies, in other words, do not easily inherit the robustness of their features. We attribute this gap in large part to the fact that the policy network must discover, from demonstrations alone, the *projective* relationship between robot actions and their projecting image features: for the policy to command the robot to reach a point observed in the image, it must

implicitly learn which actions would place the end effector there. However, this relationship between the end effector and its projecting image features is uniquely and simply determined by camera geometry: for an end effector’s position $\mathbf{x}$, the corresponding image feature lies exactly at the projection $\pi(\mathbf{x})$ given by the camera pose and intrinsics. Robot policies today are typically trained or fine-tuned on demonstrations collected from a single viewpoint, over a narrow object distribution, and in modest quantity — often yielding insufficient coverage to disentangle the projective relationship from the joint distribution over camera poses, object positions, scenes, and actions. The mapping from a 3D robot action to its corresponding image features is a single, closed-form projection equation determined by the camera pose and intrinsics, and need not be learned at all; our work instead supplies the policy with this projective relationship directly.

BIND discretizes a volume of candidate end effector positions: each 3D action candidate is attached with its projecting image features in every camera view to form an action–feature pair. Each action–feature pair is then simply scored in typical classification-supervision style (Fig. 1). We find that by making this relationship between robot actions and corresponding image features explicit, policies benefit from strong data-efficiency gains and robustness to out-of-distribution object positions and camera viewpoints.

II. RELATED WORK

a) Imitation learning: Imitation learning aims to train robot policies from demonstration data [1]–[5]: a human typically demonstrates behavior (often via teleoperation or hand-held data collection devices [6], [7]), and a visuomotor policy is supervised to produce the same actions given input observations. In its modern form, demonstrations directly serve as supervised data to learn a mapping from image observations to robot actions, without hand-designed state estimation or motion planning in between.

b) Global-regression policies: The dominant instantiation of this mapping is what we term *global regression*: image features are pooled into a single global vector, and an action head MLP decodes robot actions from it. ACT [8] regresses action chunks from a vision transformer’s CLS token with an MSE objective; Diffusion Policy [9] replaces direct coordinate regression with a denoising head; π_0 [10] and its successors decode actions with flow matching. Recent video-generation-based policy architectures likewise attach action heads that decode from a global representation alongside the predicted video [11]–[14]. Architecturally, earlier

¹University of Southern California

²Toyota Research Institute

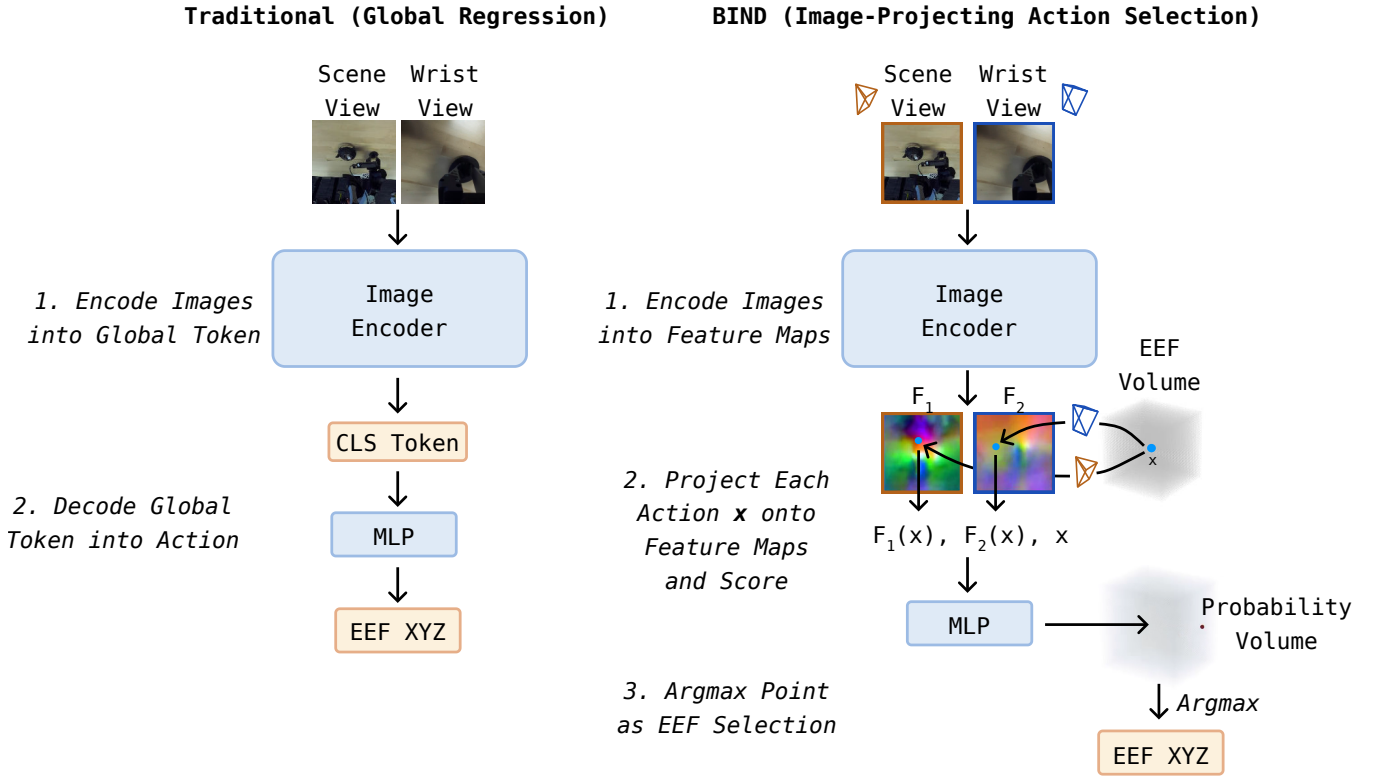


Fig. 1: **Traditional global regression vs. BIND projection-anchored selection.** Standard visuomotor policies (left) pool image features into a global descriptor and regress robot actions—forcing the network to discover the robot-action $\leftrightarrow$ pixel mapping from demonstrations. BIND (right) instead discretizes a volume of candidate end effector positions and *binds* each candidate to the image features at its projection in each camera view via known camera poses and intrinsics. Intuitively, this offers the policy the information of ‘choosing this candidate robot action would move the robot EEF to this image feature.’

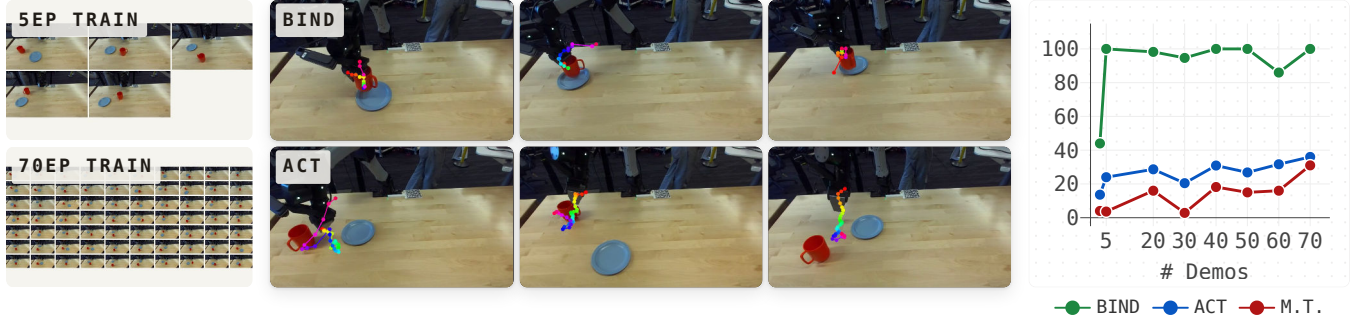
models instantiated the global pooling as the final spatial pooling layer of a CNN feature map, while modern methods use vision transformers’ dedicated CLS token or append a dedicated ‘action token’. Although these heads differ in their supervising objective—MSE, diffusion, or flow matching—they share the interface of globally pooling image features into a global token: the association between actions and the image features that ground them must still be discovered from demonstration data.

c) Voxel-categorical translation heads: PerAct [15], C2F-ARM [16], and GNFactor [17] predict end effector translation as a categorical distribution over a 3D voxel grid. This action head is similar in spirit to ours in binding robot actions to the 3D points they occupy, some of which intersect observed scene surfaces. The binding, however, is mediated by a heavyweight perception stack: RGB-D observations are voxelized into a fused 3D representation – only voxels at the surface offer image-lifted features (or often even just RGB), while the free-space voxels that most of a trajectory passes through offer none. Partly as a consequence, these methods typically predict sparse keyframe or waypoint poses rather than temporally dense actions, since the free-space trajectories offer little pretrained learning signal. BIND retains the discretize-and-classify formulation but binds *every* candidate to image features by projection (a free-space candidate still receives the features of the scene content it

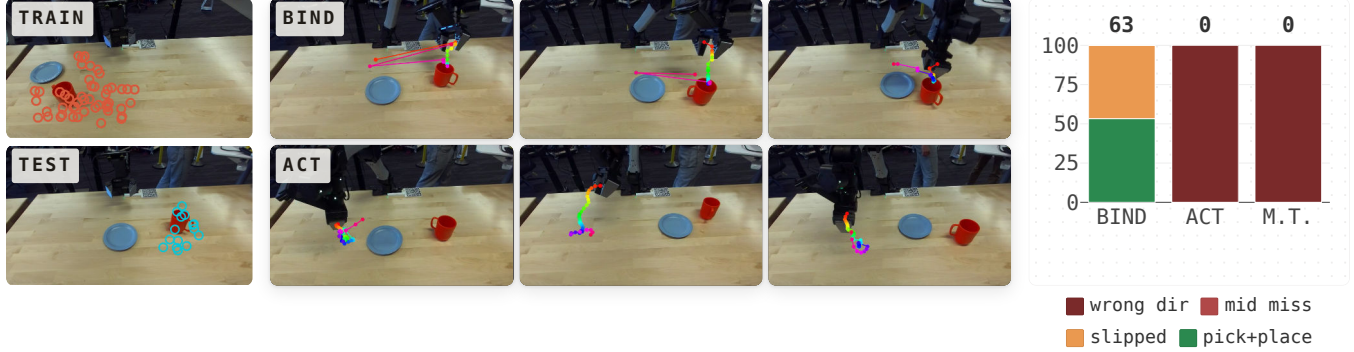
projects onto), and predicts dense-in-time action chunks. Our method also importantly does not require depth sensors and instead operates on posed-RGB images. These pipelines also often retrain their entire perception stack in the voxelized 3D space, forgoing pretrained 2D encoders and the internet-scale knowledge they embed; BIND instead fully leverages the image backbones themselves and also fully fine-tunes them jointly.

d) Rendered-view heatmap methods: RVT [18] and RVT-2 [19] also start from 3D point clouds but, rather than voxelizing them, re-render the cloud into virtual views and apply a pretrained vision encoder to the renders. The motivation behind these methods is to re-render the scene content from a fixed viewpoint to approximate being a camera viewpoint invariant policy. They similarly often also bind image features to robot actions but require a cumbersome multi-view triangulation step: an action head predicts a 2D heatmap in each view, and the 3D action is then recovered by triangulating the heatmaps on the multi-view renders. Also like the voxel-categorical methods above, these models typically output sparse keyframe waypoints. The perception stack also remains heavyweight and clunky, requiring multi-view depth sensors and point-cloud re-rendering. BIND reverses the direction of the binding: rather than predicting in 2D and lifting to 3D, it enumerates all candidate actions in 3D and binds each one *down* to its 2D image features—avoiding

Data Efficiency



Out-of-Distribution Object Position



Out-of-Distribution Viewpoint

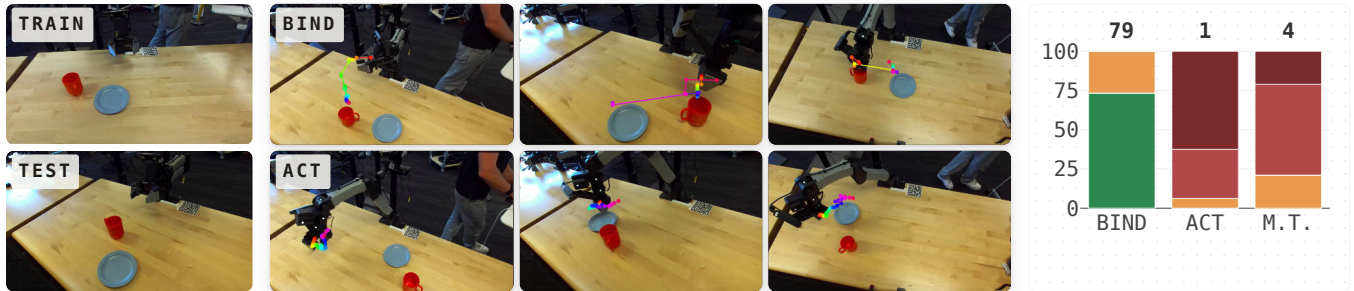


Fig. 2: **Data Efficiency and Out-of-Distribution Experiments: BIND vs. global-regression (ACT) and Motion-Tracks baselines.** Across data efficiency (top), OOD object positions (middle), and OOD camera viewpoints (bottom), BIND’s projection-anchored robot action selection outperforms the global-regression ACT and Motion Tracks baselines. All heads share the same DINOv3 backbone and training data—the only difference is the action head.

triangulation, re-rendering, and depth sensing.

e) Affordance and dense pick-and-place models.: Transporter Networks [20] and CLIPort [21] produce dense pick-and-place heatmaps and similarly bind robot actions to image features but are limited to top-down planar tasks. Affordance models [22]–[24] predict a single static interaction heatmap per image, often used as implicit guidance for downstream global-regression action models. BIND predicts full multi-step 6-DoF trajectories as feature selection over a candidate volume for each timestep, rather than a single grasp point.

III. METHOD

BIND transforms the standard global-regression action head into an image-feature-aware action head in two conceptual steps. (Fig. 4 gives a step-by-step walkthrough of the full pipeline. First, we *discretize* a space of robot actions: we discretize a world-space volume of candidate end effector

positions and cast action prediction as classification over these candidates. Second, we *bind*: before scoring each candidate robot action, we project each candidate 3D action onto every camera view to sample its corresponding image features. We concatenate the candidate position and image feature and a small classifier network scores action–feature pairs.

A. Problem Setup

We consider behavioral cloning from demonstrations $\mathcal{D} = \{(I_t^{(1)}, \dots, I_t^{(K)}, a_t, \{K_k\}, \{T_k^{\text{r} \rightarrow \text{cam}}\})\}$, where each $I_t^{(k)}$ is an RGB view, $a_t = (p_t \in \mathbb{R}^3, R_t \in \text{SO}(3), g_t \in \mathbb{R})$ is the end effector action, and $K_k, T_k^{\text{r} \rightarrow \text{cam}}$ are the per-view intrinsic and robot-to-camera extrinsic matrices. The policy predicts the next T dense-in-time end effector actions from the current observation.

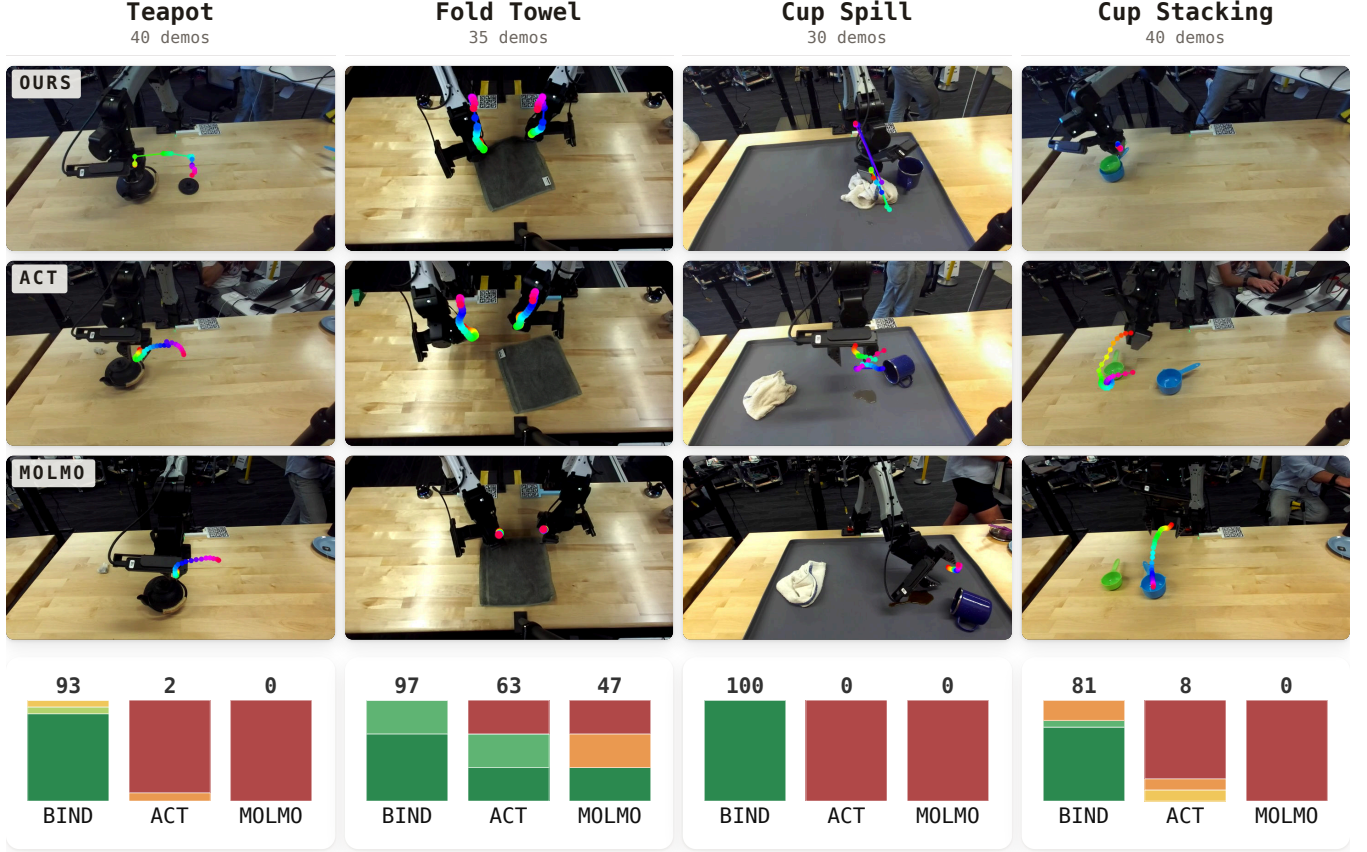


Fig. 3: **Long-horizon and dexterous real-robot tasks.** Rollouts across four tasks (left to right): teapot preparation, towel folding, spill cleanup, and cup stacking. Rows compare BIND against the ACT global-regression baseline and a fully-finetuned Molmo VLA; per-task bars report success rates for each method.

B. Step 1: Discretizing the Action Space

Discretizing the entire robot action space (e.g. joint positions) is computationally intractable. Instead, we choose to discretize a volume of candidate end effector positions, and predict rotation and gripper separately (Section III-E. While many ways exist to construct a volume of candidate locations, in practice, we construct the volume by unprojecting each pixel from each view on a lower-resolution image grid with a number of discretized height positions along each pixel’s ray.

End effector classification is predicted and supervised in a standard token-classification setup: a network predicts a $\ell_{t,v}$ for each future timestep t (Sections III-C and III-D); training minimizes cross-entropy against the candidate nearest the ground-truth position,

$$\mathcal{L}_{\text{trans}} = -\frac{1}{T} \sum_{t=1}^T \log \frac{\exp \ell_{t,v_t^*}}{\sum_v \exp \ell_{t,v}}, \quad v_t^* = \arg \min_v \|\mathbf{x}_v - p_t\|, \quad (1)$$

and inference takes the argmax, $\hat{v}_t = \arg \max_v \ell_{t,v}$, with $\hat{p}_t = \mathbf{x}_{\hat{v}_t}$.

TABLE I: RoboTwin success rate (%) across five tasks. *ManiFlow rows are quoted from their paper [25]; our evaluation modifies the simulator viewpoint, so those rows are not strictly matched to ours. ManiFlow-3D consumes point clouds; ManiFlow-2D, ACT, and Diffusion Policy use RGB and proprioception.

Method	Dual B.	Div. B.	Shoe	Cup	Apple	Mean
ManiFlow-3D*	54.0	72.3	68.3	72.7	42.0	61.9
ManiFlow-2D*	47.3	37.0	45.3	63.7	37.3	46.1
ACT	32.0	5.0	29.0	63.0	42.0	34.2
Diff. Policy	30.0	11.0	18.0	36.0	40.0	27.0
BIND (ours)	97.0	66.2	63.0	77.3	85.4	77.8

C. Step 2: Binding Actions to Image Features by Projection

For each candidate action at position $\mathbf{x} \in \mathbb{R}^3$ and each view k , we know the corresponding pixel locations via calibrated robot-camera pose and intrinsics.

$$\mathbf{u}^{(k)}(\mathbf{x}) = \pi(K_k T_k^{\text{r} \rightarrow \text{cam}} \mathbf{x}). \quad (2)$$

To obtain the corresponding image feature, we bilinearly sample the view’s spatial feature map F_k at $\mathbf{u}^{(k)}(\mathbf{x})$.

D. Scoring Action-Feature Pairs

To provide a score for each candidate action, each candidate action’s spatial position is positionally encoded and

1. Encode Posed Multiview Images into Pixel-Aligned Feature Maps
2. Sample Image Features at Each Point in EEF Volume
3. Decode into Per-Timestep Probability Volumes
4. Argmax Max-Prob EEF from Probability Volumes

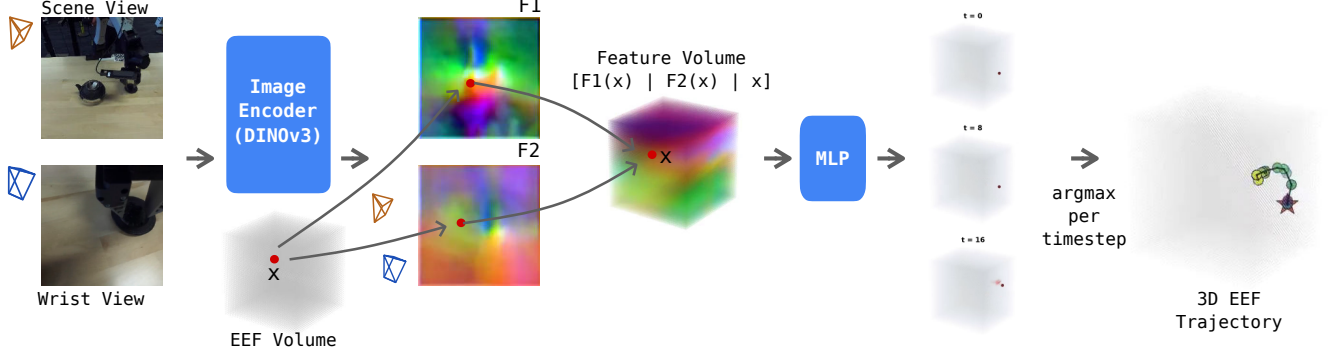


Fig. 4: **BIND forward pass.** (1) An image encoder (DINOv3) encodes the posed scene and wrist views into pixel-aligned feature maps F_1, F_2 . (2) Each candidate robot action EEF point x in the discretized EEF volume is projected to its image features from both views, forming the action-feature pair $[F_1(x) | F_2(x) | x]$. (3) An MLP decodes each action-position combination into per-timestep probability volumes. (4) The per-timestep argmax over each probability volume yields the maximum-probability 3D EEF trajectory.

TABLE II: **Real-robot cup pick-and-place.**

	BIND (ours)	ACT	Motion Tracks
<i>a) Data efficiency: score vs. training-set size</i>			
3 demos	44	14	4
5	100	24	4
20	98	29	16
30	95	21	3
40	100	31	18
50	100	27	15
60	86	32	16
70	100	36	31
<i>b) Zero-shot OOD object position: mean progress</i>			
	63	0	0
<i>c) Zero-shot OOD viewpoint (3 views pooled): mean progress</i>			
	79	1	4

TABLE III: LIBERO success rate (%) on the Spatial and Object suites. Rows sorted by the mean of the two suites.

Model	Spatial	Object	Avg
TraceVLA	84.6	85.2	84.9
OpenVLA	84.7	88.4	86.6
SpatialVLA	88.2	89.9	89.1
CoT-VLA	87.5	91.6	89.6
ThinkAct	88.3	91.4	89.9
MolmoAct-7B-D	87.0	95.4	91.2
BIND (ours)	92.0	94.0	93.0
NORA-1.5	97.3	96.4	96.9
GR00T N1.7	97.7	97.5	97.6
π_0	96.8	98.8	97.8
$\pi_{0.5}$	98.8	98.2	98.5
MolmoAct2	97.8	100.0	98.9
MolmoAct2-Think	98.8	99.8	99.3

concatenated with its corresponding image features before scoring. While one option is to provide the classifying network with the full 3D positionally-encoded coordinate, in practice, we use just the height dimension of the coordinate, since the projection already determines which pixel ray the candidate lies on. Concretely, the sampled features from all

TABLE IV: **Controlled simulation (MuJoCo cube pick-and-place).** Cells report mean progress (0–100, partial credit for touching, picking, placing, and centering the cube) / binary success %; $n=50$ episodes for in-distribution and OOD object position, $n=196$ held-out cameras for OOD viewpoint (single training camera).

	In-dist.	OOD obj.	OOD view
BIND (ours)	94/94	96/96	49/44
Diffusion-x0	82/80	0/0	12/10
MolmoAct2	81/80	0/0	12/11
ACT	45/46	0/0	12/7
DP3	23/6	0/0	8/3
Motion Tracks	12/2	0/0	4/2

TABLE V: **Long-horizon dexterous tasks (in-distribution).** Mean progress score (0–100), including the MolmoAct2 baseline (fully-finetuned per task).

	BIND	ACT	M. Tracks	MolmoAct2
Teapot (40 demos)	93	2	0	0
Cup stacking (40)	81	8	6	0
Spill cleanup (30)	100	0	3	0
Fold towel (35)	97	63	12	47

TABLE VI: **Backbone ablation (in-distribution pick-and-place).** Same BIND head and training data with the pre-trained vision encoder swapped. Progress is the mean 0–100 score; success is the binary pick-and-place rate.

Backbone	Progress	Succ. %
DINOv3	98	82
DAv3	90	88
DynaFlip	84	80
PaliGemma	88	78
π_0	78	60

views are concatenated with a positional encoding of height

and passed through an MLP,

$$\phi(\mathbf{x}) = \text{MLP}\left(\left[F_1(\mathbf{u}^{(1)}(\mathbf{x})) \parallel F_2(\mathbf{u}^{(2)}(\mathbf{x})) \parallel \text{sincos}(z)\right]\right) \in \mathbb{R}^C \quad (3)$$

where the height embedding is a 16-dimensional sin/cos positional encoding [26] of the world-height and the MLP is a small few-layer network.

Note that the image encoder is trained end-to-end with the head: we do not freeze the pretrained features. We find finetuning to be critical as it enables the features to better implicitly encode robot trajectories.

E. Rotation and Gripper Prediction

Rotation and gripper state are similarly predicted as categorical distributions over a number of discretized rotation bins and gripper bins. To predict the rotation and gripper states for each extracted 3D position keypoint, we index the cross-view fused feature $\phi(\mathbf{x}_v)$ at the ground-truth candidates during training (teacher forcing) or the argmax candidate at inference, flatten these features across the chunk’s T timesteps, and pass them through an MLP that outputs the rotation and gripper trajectories for the whole chunk.

IV. EXPERIMENTS

We evaluate BIND across three axes—data efficiency, robustness to OOD object positions, and robustness to OOD camera viewpoints—and additionally on long-horizon dexterous tasks. We perform these studies in the real-world as well as simulation for reproducibility and precise definitions of camera and object distributions. Since BIND is a general action head, we also demonstrate performance with a number of different backbones, including pure-vision backbones as well as vision-language models.

a) Baselines: *ACT* [8] (CLS→XYZ): the CLS token of the shared backbone is fed to an MLP that regresses end effector coordinates and gripper state directly—the standard global-regression head. *Motion Tracks* (CLS→UVZ) [27]: a modified version of Motion Tracks that predicts 2D image-space tracks plus ray-height from the same global token and recovers end effector coordinates from them (instead of using two views for triangulation from the global pixel coordinate regression). *MolmoAct2* [28]: a 7B-parameter vision-language-action model, fully-finetuned per task on the same demonstrations. The controlled simulation study additionally compares against Diffusion Policy [9] and the point-cloud-based DP3 [29], and the RoboTwin benchmark against their reported ManiFlow [25].

b) Setup: We use YAM arms for experiments with a scene camera as well as a wrist camera. We obtain the camera pose via a Fiducial Exoskeletons [30] style calibration board. All methods use a shared DINOv3 [31] ViT-S/16+ backbone, are trained from a single viewpoint with no data augmentation, and are scored by mean task progress (0–100, weighted by outcome category, rather than binary success).

c) Tasks: For the core experiments tasks on data-efficiency and robustness, we use a simple pick-and-place task of picking up a cup and placing it on a plate. For testing more dexterous and long horizon tasks, we use four more tasks: The teapot task involves taking off the lid of a teapot, placing a teabag in the teapot, and putting the lid back on the teapot; the lid also has a thin grasp point extrusion which requires high precision. The cup stacking task requires the robot to pick and place a smaller measuring cup into another larger measuring cup in the correct orientation of the larger measuring cup. The cup-spill task requires the robot to reorient the fallen cup, pick the rag up, and wipe the liquid spill. The fold-towel is a bi-manual task that requires folding a flat towel cleanly in half.

A. Data Efficiency

Our first claim is that BIND yields strong data-efficiency gains. We sweep the pick-and-place training set from 3 demonstrations up to 70 (Fig. 2; Tab. II). BIND sustains a progress score of 86–100 at every training-set size of 5 episodes or more—reaching 100 (10/10 successes) with as few as 5 demonstrations—and degrades to 44 only at 3 episodes. The ACT baseline never exceeds 36% at any data size, and Motion Tracks never exceeds 31%, as we randomize the positions of the cup during training and evaluation.

B. Out-of-Distribution Object Position

We claim that BIND is less sensitive to the train and test distributions alignment of object positions. To demonstrate this, we train the cup place task only on the left half of the table, and then at evaluation move the cup to the right half of the table zero-shot (Fig. 2; Tab. II). BIND attains a score of 63 on held-out placements; ACT and Motion Tracks both collapse to 0. Note that our method is not invariant to the position of the cup – in fact, our method degrades significantly (from 98 in-distribution to 63). Our claim here is that our method degrades significantly more gracefully and is more *robust* to the out-of-distribution zero-shot setting: while our method does not achieve the same precision as the in-distribution manipulation, when it fails it at least is a ‘close’ failure of reaching near the object, whereas the baselines do not even move towards the correct side of the table.

C. Out-of-Distribution Camera Viewpoint

Our third claim is that BIND is less sensitive to the camera position. We train at just one camera position and evaluate at three different camera positions. BIND attains a score of 79 aggregated across the three views (Tab. II); ACT and Motion Tracks collapse to 1 and 4. Once again, our claim is not of *invariance* to the camera position, but rather that small shifts in camera distribution are of almost no consequence, and extreme camera position changes (e.g. switching to a near birds-eye view) still behave reasonably and even often succeed.

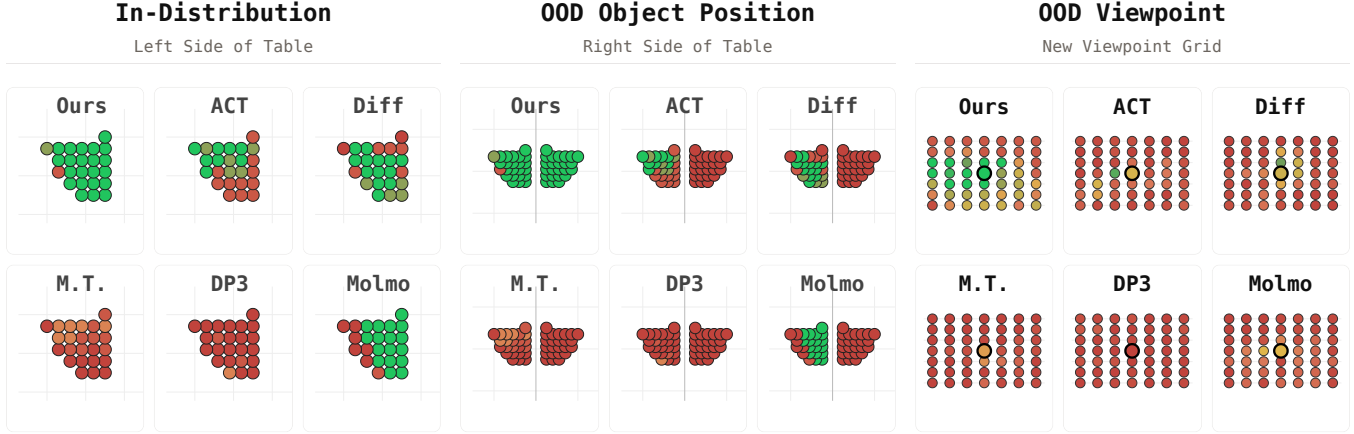


Fig. 5: **Controlled simulation experiments (MuJoCo cube pick-and-place).** We study data efficiency and OOD object position and OOD viewpoints with precise distribution definitions on a controlled MuJoCo cube pick-and-place setup. We compare BIND against five baselines (Diffusion Policy, MolmoAct2, ACT, Motion Tracks, and DP3). BIND is the only method that retains meaningful progress under both distribution shifts.

D. Long-Horizon Dexterous Tasks

We extend beyond simple object pick-and-place to four multi-stage tasks (Fig. 3; Tab. V). *Teapot* (40 demonstrations): remove the teapot lid, place a teabag inside, and close the lid—BIND 93 vs. ACT 2. *Stack cup* (40 demonstrations): pick the smaller measuring cup and stack it onto the larger measuring cup in the same orientation—BIND 81 vs. ACT 8. *Spill cleanup* (30 demonstrations): reorient a spilled cup upright, pick up a rag, wipe the table—BIND 100 vs. ACT 0. *Fold towel* (35 demonstrations): fold a flat towel cleanly in half with both arms—BIND 97 vs. ACT 63. The fully-finetuned MolmoAct2 scores 0 on the three cup-and-teapot tasks and 47 on fold towel; Motion Tracks never exceeds 12. Our method succeeds on these more challenging, precise, and long-horizon tasks with a relatively small number of demonstrations (even just 30 examples).

E. Backbone Ablation

Because our action head requires only pixel-aligned feature maps, our method is plug-and-play and the pretrained encoder is swappable (Tab. VI). We run the same in-distribution pick-and-place task with five encoders spanning self-supervised, geometric, robotics, VLM, and VLA families [10], [31]–[34]: DINOv3 82%, DepthAnything v3 88%, DynaFlip 80%, PaliGemma 78%, and π_0 60%. We observe that the spatial features from the general vision encoders (especially DINO) have much qualitatively smoother feature maps than the VLM family of backbones and yield the best results.

F. Controlled Simulation Analysis

Measuring the exact object position and camera viewpoint distributions can be challenging in the real-world, so we also offer study in simulation (Tab. IV). We replicate the two OOD studies in a controlled MuJoCo [35] cube pick-and-place environment where we carefully define the object

and camera distributions, comparing against five baselines on identical data: ACT, Diffusion Policy, MolmoAct2, Motion Tracks, and the point-cloud-based DP3 (Fig. 5).

a) *Position OOD.*: Training positions cover only the left half of the workspace; testing covers held-out mirrored positions on the unseen right half. In distribution, BIND reaches a score of 94, ahead of the strongest baselines Diffusion Policy (82) and MolmoAct2 (81). On the held-out half, BIND retains 96 while every baseline drops to 0.

b) *Viewpoint generalization.*: We train from a single camera and evaluate zero-shot across a grid of 196 held-out viewpoints. BIND maintains a score of 49, while all baselines score 12 or lower. As in the real-world study, BIND is not viewpoint-invariant, but it degrades gracefully where global-regression heads fail outright.

G. LIBERO Benchmark

We also evaluate BIND on the LIBERO [36] Spatial and Object suites (Tab. III). BIND is trained from scratch on each suite and reaches 92.0% on Spatial and 94.0% on Object—above every sub-95 7B baseline on at least one suite and within a few points of the large-scale pretrained frontier.

H. RoboTwin Benchmark

We also evaluate BIND on five tasks from the RoboTwin bimanual simulation benchmark [37] (Table I). BIND reaches a mean success rate of 77.8%, ahead of the strongest baseline, point-cloud-based ManiFlow-3D [25] (61.9), while operating from RGB alone.

V. CONCLUSION

We present BIND, a projection-anchored action formulation that binds each candidate end effector position to the 2D image features at its projection. In real-world and simulation settings, BIND reaches strong pick-and-place success from as few as 5 demonstrations, is robust to OOD

object positions and camera viewpoints where coordinate-regression baselines fail completely, and completes long-horizon dexterous tasks with as few as 30–40 demonstrations. Controlled simulation experiments replicate both robustness results.

REFERENCES

- [1] S. Schaal, “Learning from demonstration,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 1996.
- [2] C. G. Atkeson and S. Schaal, “Robot learning from demonstration,” in *International Conference on Machine Learning (ICML)*, 1997, pp. 12–20.
- [3] S. Levine, C. Finn, T. Darrell, and P. Abbeel, “End-to-end training of deep visuomotor policies,” *Journal of Machine Learning Research*, vol. 17, no. 39, pp. 1–40, 2016, arXiv:1504.00702.
- [4] F. Torabi, G. Warnell, and P. Stone, “Behavioral cloning from observation,” in *International Joint Conference on Artificial Intelligence (IJCAI)*, 2018, arXiv:1805.01954.
- [5] Y. Duan, *et al.*, “One-shot imitation learning,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017, arXiv:1703.07326.
- [6] P. Wu, Y. Shentu, Z. Yi, X. Lin, and P. Abbeel, “GELLO: A general, low-cost, and intuitive teleoperation framework for robot manipulators,” *arXiv preprint arXiv:2309.13037*, 2023.
- [7] C. Chi, *et al.*, “Universal manipulation interface: In-the-wild robot teaching without in-the-wild robots,” *arXiv preprint arXiv:2402.10329*, 2024.
- [8] T. Z. Zhao, V. Kumar, S. Levine, and C. Finn, “Learning fine-grained bimanual manipulation with low-cost hardware,” in *Robotics: Science and Systems (RSS)*, 2023, arXiv:2304.13705.
- [9] C. Chi, *et al.*, “Diffusion policy: Visuomotor policy learning via action diffusion,” in *Robotics: Science and Systems (RSS)*, 2023, arXiv:2303.04137.
- [10] K. Black, *et al.*, “ π_0 : A vision-language-action flow model for general robot control,” in *Robotics: Science and Systems (RSS)*, 2025, arXiv:2410.24164.
- [11] Y. Du, *et al.*, “Learning universal policies via text-guided video generation,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023, arXiv:2302.00111.
- [12] S. Li, Y. Gao, D. Sadigh, and S. Song, “Unified video action model,” *arXiv preprint arXiv:2503.00200*, 2025.
- [13] M. J. Kim, *et al.*, “Cosmos policy: Fine-tuning video models for visuomotor control and planning,” *arXiv preprint arXiv:2601.16163*, 2026.
- [14] S. Ye, *et al.*, “World action models are zero-shot policies,” *arXiv preprint arXiv:2602.15922*, 2026.
- [15] M. Shridhar, L. Manuelli, and D. Fox, “Perceiver-actor: A multi-task transformer for robotic manipulation,” in *Conference on Robot Learning (CoRL)*, 2022, arXiv:2209.05451.
- [16] S. James, K. Wada, T. Laidlow, and A. J. Davison, “Coarse-to-fine q-attention: Efficient learning for visual robotic manipulation via discretisation,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022, arXiv:2106.12534.
- [17] Y. Ze, *et al.*, “Gnfactor: Multi-task real robot learning with generalizable neural feature fields,” in *Conference on Robot Learning (CoRL)*, 2023, arXiv:2308.16891.
- [18] A. Goyal, J. Xu, Y. Guo, V. Blukis, Y.-W. Chao, and D. Fox, “Rvt: Robotic view transformer for 3d object manipulation,” in *Conference on Robot Learning (CoRL)*, 2023, arXiv:2306.14896.
- [19] A. Goyal, V. Blukis, J. Xu, Y. Guo, Y.-W. Chao, and D. Fox, “Rvt-2: Learning precise manipulation from few demonstrations,” in *Robotics: Science and Systems (RSS)*, 2024, arXiv:2406.08545.
- [20] A. Zeng, *et al.*, “Transporter networks: Rearranging the visual world for robotic manipulation,” in *Conference on Robot Learning (CoRL)*, 2020, arXiv:2010.14406.
- [21] M. Shridhar, L. Manuelli, and D. Fox, “Cliport: What and where pathways for robotic manipulation,” in *Conference on Robot Learning (CoRL)*, 2021, arXiv:2109.12098.
- [22] Y. Tang, *et al.*, “UAD: Unsupervised affordance distillation for generalization in robotic manipulation,” *arXiv preprint arXiv:2506.09284*, 2025.
- [23] S. Nasiriany, *et al.*, “RT-Affordance: Affordances are versatile intermediate representations for robot manipulation,” *arXiv preprint arXiv:2411.02704*, 2024.
- [24] Y. Kuang, *et al.*, “RAM: Retrieval-based affordance transfer for generalizable zero-shot robotic manipulation,” *arXiv preprint arXiv:2407.04689*, 2024.
- [25] G. Yan, *et al.*, “Maniflow: A general robot manipulation policy via consistency flow training,” *arXiv preprint arXiv:2509.01819*, 2025.
- [26] A. Vaswani, *et al.*, “Attention is all you need,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017, arXiv:1706.03762.
- [27] J. Ren, P. Sundaresan, D. Sadigh, S. Choudhury, and J. Bohg, “Motion tracks: A unified representation for human-robot transfer in few-shot imitation learning,” *arXiv preprint arXiv:2501.06994*, 2025.
- [28] H. Fang, *et al.*, “Molmoact2: Action reasoning models for real-world deployment,” *arXiv preprint arXiv:2605.02881*, 2026.
- [29] Y. Ze, G. Zhang, K. Zhang, C. Hu, M. Wang, and H. Xu, “3d diffusion policy: Generalizable visuomotor policy learning via simple 3d representations,” in *Robotics: Science and Systems (RSS)*, 2024, arXiv:2403.03954.
- [30] C. Smith, B. Van Hoorick, V. Guizilini, and Y. Wang, “Fiducial exoskeletons: Image-centric robot state estimation,” *arXiv preprint arXiv:2601.08034*, 2026.
- [31] O. Siméoni, *et al.*, “DINOv3,” *arXiv preprint arXiv:2508.10104*, 2025.
- [32] H. Lin, *et al.*, “Depth anything 3: Recovering the visual space from any views,” *arXiv preprint arXiv:2511.10647*, 2025.
- [33] J. Lee, *et al.*, “DynaFLIP: Rethinking robotics perception via tri-modal-dynamics guided representation,” *arXiv preprint arXiv:2605.30350*, 2026.
- [34] L. Beyer, *et al.*, “PaliGemma: A versatile 3B VLM for transfer,” *arXiv preprint arXiv:2407.07726*, 2024.
- [35] E. Todorov, T. Erez, and Y. Tassa, “MuJoCo: A physics engine for model-based control,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2012.
- [36] B. Liu, *et al.*, “LIBERO: Benchmarking knowledge transfer for lifelong robot learning,” *arXiv preprint arXiv:2306.03310*, 2023.
- [37] Y. Mu, *et al.*, “Robotwin: Dual-arm robot benchmark with generative digital twins,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025, arXiv:2504.13059.